

Cloud-Native Detection of Cyberattacks Against Satellite Ground Stations: An Open, Labelled Testbed and Two Negative Results

Henrique

Independent researcher

<https://github.com/rikosoo/Cloud-Native-Detection-of-Cyberattacks-Against-Satellite-Ground-Stations>

Abstract—The space segment of a satellite mission is expensive to reach; the ground segment that commands it is an ordinary cloud workload with operators, credentials, APIs and object storage. Published satellite-security research has concentrated on the spacecraft and the radio link, and there is no open, labelled corpus against which ground-segment detection can be measured. We present an open testbed that closes that gap: a simulated satellite ground station that emits four correlated planes of evidence (spacecraft telemetry, authenticated telecommands, cloud audit records and payload downlinks), five adversary-emulation scenarios mapped to SPARTA and MITRE ATT&CK with per-event ground truth, and a cloud-native detection pipeline of 21 stateful rules plus an unsupervised telemetry baseline, deployed on AWS through Kinesis, Lambda, DynamoDB, CloudTrail, GuardDuty and Security Hub. Over 20 independent 24-hour trials the pipeline detects all five scenarios in every trial at a precision of 0.997 ± 0.002 , with a false-positive rate of 0.06% of benign events. The learned baseline is what makes a sub-threshold behavioural drift visible: it fires 42 ± 2 minutes before the first spacecraft limit violation, and raises event-level recall for that scenario from 0.19 to 0.97. Two results contradict the design rationale we started from. First, the multivariate model does not beat a well-calibrated per-channel detector on that drift (AUC 0.996 ± 0.001 versus 0.998 ± 0.000). Second, a controlled probe that preserves every marginal distribution while destroying the joint structure — replaying housekeeping frames from half an orbit earlier — is invisible to both detectors (AUC 0.322 and 0.385, detected in 1 of 20 trials), because a baseline fitted over a whole orbit marginalises away the very phase information the attack violates. We also find a sharp operational threshold: fitting the baseline on less than a full diurnal cycle raises the telemetry false-positive rate from 0.2% to 77%. All code, data and the experiment harness are public.

Index Terms—satellite ground stations, space systems security, intrusion detection, anomaly detection, cloud security, adversary emulation, telemetry

I. INTRODUCTION

A satellite is difficult to attack directly and difficult to repair once attacked. The ground station that commands it is neither. Modern ground segments are built from the same components as any other cloud workload: identity providers, API endpoints, message queues, object storage and the operators who hold the credentials. An adversary who reaches the commanding path from there inherits authority over the vehicle without ever touching a radio.

Recent work has made the space segment measurable. Willbold et al. [1] performed the first systematic software-security analysis of real satellite firmware; Pavur et al. demonstrated large-scale interception of operational satellite links [2] and argued for a research agenda with reproducible artefacts [3]. Policy has followed, with Space Policy Directive-5 naming the ground segment explicitly as part of the protected system [4], and the community now has a space-specific adversary taxonomy in SPARTA [5].

What is missing is the thing that made progress possible in network intrusion detection: an open corpus with ground truth. Surveys of network IDS datasets list dozens of options [6]; for the satellite ground segment there is, to our knowledge, none. Ground-segment telemetry and audit logs are operationally sensitive and rarely published, and when they are, they are unlabelled. The practical consequence is that ground-segment detection claims cannot be compared, and that a detector's cost — in false positives, in analyst time, in compute — is usually not reported at all.

This paper contributes a testbed and an evaluation rather than a new detector. Concretely:

- **An open, labelled ground-station corpus generator** (Section IV). A simulated station emits four correlated planes of evidence, and five adversary-emulation scenarios — credential compromise, telecommand tampering, replay, sub-threshold behavioural drift and bulk exfiltration — inject themselves into that timeline, labelling every event they create or modify. Scenarios compose: the stolen session established by the credential stage is the one the tampering and exfiltration stages reuse.
- **A cloud-native detection pipeline** (Section III). 21 stateful rules covering identity, commanding integrity, replay, spacecraft limits and exfiltration, each mapped to an ATT&CK technique [7] and emitted in the AWS Security Finding Format [8], plus an unsupervised telemetry baseline that runs inside a Lambda function with no machine-learning runtime. The same engine object runs offline, under test, and in the cloud.
- **A measured evaluation with negative results** (Sections V–VI). Twenty independent trials, an ablation sep-

arating rules from the model, a comparison against the two alternatives a real mission already has (a per-channel detector and the spacecraft’s own limit checks), a probe that isolates joint structure from marginals, a baseline-size sensitivity sweep, and the compute cost. Two of the findings contradict the rationale we designed the model around, and we report them as such.

We organise the evaluation around five questions:

- RQ1 Does the pipeline detect each scenario, and at what precision?
- RQ2 What does the learned baseline add over the spacecraft’s own limit checks — the alternative a real mission already operates?
- RQ3 Does the multivariate model add anything over a well-calibrated per-channel detector?
- RQ4 How much clean telemetry does the baseline need before it is usable?
- RQ5 What does the pipeline cost to run?

II. BACKGROUND AND THREAT MODEL

A. The ground segment as an attack surface

A ground station converts operator intent into authenticated telecommands and spacecraft state into archived data products. Four kinds of record are produced, and a detector needs all four:

- 1) *Telemetry*. Spacecraft housekeeping — bus voltage and current, battery state of charge, amplifier and battery temperatures, link margin — downlinked continuously during a contact.
- 2) *Telecommands*. CCSDS TC frames [9], authenticated in modern missions by the Space Data Link Security protocol [10], which supplies a keyed authentication tag and an anti-replay sequence counter.
- 3) *Cloud audit records*. Control-plane activity: who signed in, from where, with what authenticator, and which APIs they called.
- 4) *Downlink transfers*. Payload products moving into (and potentially out of) the mission archive.

The first two describe the vehicle; the last two describe the enterprise. Most published space-security work addresses the first two, and most enterprise security tooling addresses the last two. An intrusion crosses between them, which is the observation the design in Section III is built on.

B. Adversaries

We consider three profiles, following the taxonomies in [11] and [12]:

Opportunistic. Credential stuffing, commodity tooling, no mission knowledge. High volume, low stealth. Interested in compute and in ransoming the archive.

Insider. A badged operator acting outside their role. Uses legitimate credentials from legitimate networks at legitimate times; only *what* they do is anomalous. Defeats every network- and identity-provenance rule by construction.

State-sponsored. Patient, mission-aware, targets key material and the commanding path, disables logging early, and prefers degradation that is indistinguishable from a bus fault — because an operator who believes the spacecraft is unhealthy will act on the wrong runbook.

C. Trust boundaries and attack paths

Four boundaries separate the operator from the vehicle: the human-to-cloud boundary (B1, defended by multi-factor authentication and network policy), the cloud control plane (B2, defended by least-privilege authorisation and audit logging), the uplink (B3, defended by the authentication tag and the sequence counter) and the spacecraft itself (B4, defended by on-board limit checks). Table I lists the five attack paths we emulate, the boundary each crosses, and its mapping to SPARTA [5] and ATT&CK [7].

D. Assumptions

We assume the event stream reaches the detector: an adversary who can silently suppress telemetry defeats every rule, which is why the deployed pipeline alarms on the *absence* of events. We assume audit logging is on, and treat its disabling as itself a critical finding. We assume telecommand key material is not already stolen; when it is, the integrity rule goes quiet and only the behavioural rules remain, which is why the catalogue does not rest on the authentication tag alone.

III. SYSTEM DESIGN

Figure 1 shows the pipeline. Three properties drove the design.

One engine, three runtimes. The detection engine imports no cloud SDK. It runs unchanged in the offline command-line tool, in the test suite, and inside an AWS Lambda function. Cloud services are reached only through an event sink interface and a state-store interface.

Offline first. The full pipeline — generate, fit, detect, score — runs with NumPy and a YAML parser, with no credentials and no network. This is what makes the artefact reproducible by a reader who has no AWS account.

Measured, not asserted. Every injected event carries a ground-truth label, so precision, recall and detection latency are computed rather than claimed.

A. Event envelope

All four planes share one envelope: an identifier, a type, a UTC timestamp, the station identifier, a plane-specific payload, and (in the lab only) the ground-truth label. A single stream means a single consumer, and the audit payload deliberately mirrors the field names of a real cloud trail so that the identity rules run unchanged against production logs.

B. Rule engine

The engine consumes events in timestamp order and emits findings carrying a severity, an ATT&CK technique, the evidence that triggered them and a remediation. Severities and thresholds live in a YAML catalogue, separate from the logic, so an operator can retune without a code deployment.

TABLE I
EMULATED ATTACK PATHS, THE TRUST BOUNDARY EACH CROSSES, AND FRAMEWORK MAPPING.

Scenario	Boundary	SPARTA	ATT&CK	Adversary action
credential_compromise	B1, B2	IA-0004	T1110.003, T1078.004, T1098.001	Password spray, sign-in without MFA from an untrusted network outside duty hours, new access key, administrative policy attachment, audit trail deleted.
command_tampering	B3	EX-0012	T1565.002	Opcode and parameters rewritten in flight; an authentication tag forged with the wrong key; an out-of-role command; an uplink flood.
replay	B3	EX-0013	T1557	Recorded frames retransmitted verbatim hours later. The tag still verifies: the adversary never held the key [13].
anomalous_behavior	B4	IMP-0002	T1565.001	Commanded degradation in which every channel stays inside its interface-control limits while the relationships between them break.
exfiltration	B2	EXF-0003	T1530, T1537, T1567.002	Archive enumeration, bulk object reads, multi-gigabyte cross-account copy to an unapproved destination.

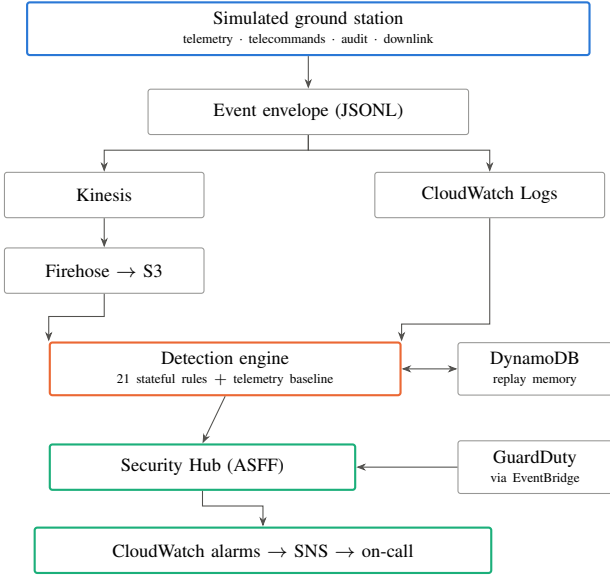


Fig. 1. Pipeline. The detection engine is the same object offline, under test and inside Lambda; cloud services are reached only through the sink and state-store interfaces. GuardDuty findings are re-scored against the mission asset inventory before entering the same queue.

State is bounded by time rather than by event count. Three horizons appear: failed-login, command-rate and anomaly-streak state over seconds to minutes; last-login geography per operator over an hour; and accepted sequence counters and authentication tags over the full anti-replay window of 24 hours.

Only the last horizon must be globally consistent. A replayed frame was accepted hours earlier, quite possibly by a different Lambda invocation on a different shard, so replay detection is the one rule that cannot be answered from a single batch. The engine therefore addresses a *replay memory* interface — a dictionary offline, and in the cloud a DynamoDB table with a time-to-live and a conditional write, which makes the deduplication race-free without a lock.

C. Telemetry baseline

Let $x \in \mathbb{R}^d$, $d = 7$, be the vector of housekeeping channels. We fit a single robust Gaussian on telemetry from a run known to be clean.

Location and scale are estimated per channel by the median and the normalised median absolute deviation [14], so that the few outliers any real archive contains do not drag the baseline:

$$\hat{\mu}_j = \text{med}(x_j), \quad \hat{\sigma}_j = 1.4826 \cdot \text{med}(|x_j - \hat{\mu}_j|). \quad (1)$$

Channels are standardised, $z = (x - \hat{\mu}) \oslash \hat{\sigma}$, and the sample covariance S of z is shrunk towards its diagonal in the spirit of [15] with a fixed intensity $\lambda = 0.15$, so that the inverse remains well conditioned on short baselines:

$$\tilde{S} = (1 - \lambda)S + \lambda \text{diag}(S). \quad (2)$$

The anomaly score of a sample is its squared Mahalanobis distance [16],

$$s(x) = z^\top \tilde{S}^{-1} z, \quad (3)$$

and the detection threshold τ is the empirical q -quantile of s over the training run, with $q = 0.999$ throughout this paper. A finding is raised only after $k = 3$ consecutive samples exceed τ ; Section VI shows that this streak requirement is what converts a non-zero per-sample false positive rate into zero false alerts.

Three properties matter more than accuracy here. The model fits in milliseconds, serialises to a JSON object of 1.5 KB that a Lambda function loads without a machine-learning layer, and decomposes: the per-channel contributions to (3) are available for free, so a finding can name the channels that moved. An analyst at 03:00 needs that more than a fractionally better score, and Section VI shows the fractionally better score was not on offer anyway.

IV. IMPLEMENTATION

A. Spacecraft and station model

The simulated spacecraft is a 95-minute low-Earth orbit with a 35% eclipse fraction and an 11-minute contact window per orbit. The bus model is small but deliberately *coupled*: illumination drives the power budget, the power budget and

TABLE II
DEPLOYMENT MAPPING.

Stage	Service
Ingestion	Kinesis Data Streams (station-partitioned)
Raw archive	Firehose → S3, GZIP, tiered to Glacier
Detection	Lambda, ReportBatchItemFailures
Replay memory	DynamoDB, conditional write + TTL
Identity plane	CloudTrail, management + S3 data events
Native detection	GuardDuty, re-scored by a correlator Lambda
Finding store	Security Hub (ASFF [8])
Alerting	CloudWatch embedded metrics, alarms, SNS

eclipse state drive the thermal channels, and transmit duty and slew rate drive the amplifier temperature and the link margin. Coupling is what makes the question in RQ3 meaningful — an adversary who moves one channel while leaving the others untouched breaks the relationships long before it breaks any single limit.

The station drives that model at a 30-second cadence and adds the three ground-segment planes: shift handovers produce console sign-ins (service identities federate through role assumption instead), contact windows produce telecommands authenticated with a keyed tag over a canonical serialisation plus a monotonic sequence counter, and each pass ends with a payload downlink to the mission archive.

B. Attack injectors

Attacks are applied *after* the nominal run, so “normal” is defined in one place. Each injector receives the timeline and a context object, and returns the timeline with its events inserted or mutated and labelled. This design has two consequences we rely on. Injectors compose — the credential stage registers the hijacked session that the tampering and exfiltration stages reuse, which is what produces genuine cross-plane correlation rather than five independent incidents. And because injection is a transformation of a clean timeline, the same seed yields the same corpus, byte for byte.

C. Cloud deployment

Table II maps each stage to the service that carries it. The infrastructure is defined as code; the detector and the GuardDuty correlator are Lambda functions sharing a layer built from the same package the offline tool imports.

Three defects appeared only once the cloud path was exercised against emulated services, and we record them because each is a silent failure of the kind a simulation-only evaluation would never surface. First, the log-ingestion API rejects events older than 14 days *inside a successful response*: the reproducible corpus, whose timestamps are fixed for determinism, vanished into an empty log group with a zero exit code. Second, declaring advanced event selectors on the audit trail *replaces* the default management-event selector, so a data-events-only configuration would have recorded no sign-ins or credential creations at all, silently disabling the entire identity rule family. Third, the stream-to-function mapping advertises partial-batch failure reporting, but a handler that does

TABLE III
ONE 24-HOUR CORPUS: EVENTS BY PLANE, AND EVENTS CARRYING EACH ATTACK LABEL. LABELLED EVENTS OVERLAP THE PLANES THEY OCCUR IN.

	Events	%
<i>Evidence plane</i>		
telemetry	2880	80.4
telecommand	583	16.3
audit	102	2.8
downlink	19	0.5
<i>Attack label</i>		
credential_compromise	11	0.3
command_tampering	43	1.2
replay	8	0.2
anomalous_behavior	404	11.3
exfiltration	7	0.2
Benign	3111	86.8
Total	3584	100.0

not return the failed record identifiers causes one malformed record to fail and retry an entire 200-record batch. All three are fixed and covered by regression tests.

V. EXPERIMENTAL METHODOLOGY

A. Corpus

Table III describes one 24-hour run. Telemetry dominates by construction, at one sample every 30 seconds; the identity and downlink planes are sparse, which is exactly the regime in which a per-event false positive rate matters.

The label distribution is uneven and we do not smooth it: replay and exfiltration label single-digit numbers of events, while anomalous_behavior labels several hundred telemetry samples because the drift it injects persists for hours. Aggregate recall would therefore be dominated by one scenario, so we report recall per scenario throughout and never average it.

B. Protocol

We run 20 independent trials. Trial i fits the telemetry baseline on a clean 24-hour run generated with seed $i + 10^4$, then detects on an attacked run generated with seed i . Training and evaluation data are therefore disjoint and independently generated, and no attacked sample ever reaches the model. All trials use the same rule catalogue and the same $q = 0.999$ threshold quantile; nothing is tuned per trial or per scenario.

C. Metrics

A finding is a *true positive* when the event it names carries an attack label, and a *false positive* otherwise. From this we report:

- **Detection rate**: the fraction of trials in which a scenario produced at least one true positive. This is the question an operator asks.
- **Event-level recall**: within a scenario, the fraction of its labelled events that produced at least one finding. This is the stricter question, and the one where a persistent

scenario can be “detected” while most of its events go unflagged.

- **Precision and false-positive rate**, the latter over benign events rather than over findings, so that it is comparable across scenarios of different sizes.
- **Mean time to detect (MTTD)**: the interval between a scenario’s first labelled event and the first finding attributable to it.

D. Ablation and reference detectors

We evaluate three configurations: *rules only* (the catalogue with the model-driven rule removed), *model only* (that rule alone), and *rules + model*. For RQ2 and RQ3 the model is compared against the two alternatives a real mission already has:

Interface-control limits. The spacecraft’s own red lines, encoded as a rule. This is the deployed status quo, and the only honest comparison for “does the learned baseline earn its place”.

Best single channel. A per-channel detector that already knows the robust centre and scale of every channel and takes the largest absolute standardised deviation, $u(x) = \max_j |z_j|$, thresholded at the same training quantile. This is intentionally the strongest univariate strawman: it is given everything the multivariate model has except the covariance.

E. The phase-swap probe

The drift injected by `anomalous_behavior` pushes several channels well outside their own robust scale, which means it is detectable marginally and cannot separate “multivariate” from “well-calibrated univariate”. To test the design claim directly we construct a probe that preserves every marginal exactly while destroying the joint structure.

In a clean run, each telemetry vector in a 50-minute window is replaced by the vector observed exactly half an orbit earlier, channel for channel. Every individual value is one the spacecraft genuinely produced and lies inside its own learned marginal distribution; the *combination* — eclipse-era power draw beside sunlit-era temperatures — is physically impossible. Orbit phase is not among the model’s features, so the substitution is invisible to any per-channel test by construction, and detectable only by a model of the relationships between channels.

The probe also models a real technique: replaying recorded housekeeping frames to mask a fault or the effect of a malicious command. We keep it as a controlled probe rather than a sixth deployed scenario so that the headline effectiveness numbers describe the shipped system.

F. Environment

All experiments run single-threaded on the container described in the artefact’s `results.json`, which records the Python and NumPy versions and the platform string alongside every number reported here. The harness is deterministic: re-running it reproduces every figure and every value in this paper.

TABLE IV
EFFECTIVENESS OVER 20 INDEPENDENT TRIALS. DETECTION RATE IS THE PERCENTAGE OF TRIALS IN WHICH THE SCENARIO PRODUCED AT LEAST ONE FINDING; RECALL IS PER LABELLED EVENT. THE LAST TWO COLUMNS GIVE RECALL FOR THE ABLATED CONFIGURATIONS.

Scenario	Det.	Recall	Recall (ablated)		MTTD (s)
	%	rules + model	rules	model	
<code>credential_compromise</code>	100	0.82 ± 0.00	0.82	0.00	18
<code>command_tampering</code>	100	1.00 ± 0.00	1.00	0.00	0
<code>replay</code>	100	1.00 ± 0.00	1.00	0.00	0
<code>anomalous_behavior</code>	100	0.97 ± 0.00	0.19	0.97	342
<code>exfiltration</code>	100	1.00 ± 0.00	1.00	0.00	0
Precision (rules + model): 0.997 ± 0.002					
False positives: 0.06% of benign events					

VI. RESULTS

A. RQ1: Effectiveness

Table IV and Figure 2 give the main result. All five scenarios are detected in all 20 trials, at a precision of 0.997 ± 0.002 and a false-positive rate of 0.06% of benign events. The four scenarios that leave discrete evidence — a forged tag, a reused counter, a policy attachment, a transfer to an unapproved destination — reach event-level recall of 1.00 with zero variance and an MTTD of 0 seconds, because the first event of the scenario is itself the violation.

The ablation shows the two halves are complementary and non-overlapping. The model alone detects nothing outside the telemetry plane — it has nothing to say about a forged authentication tag — and the rules alone reach only 0.19 recall on the behavioural drift. Together they cover the corpus. The model configuration attains a precision of 1.000 with zero false positives in any trial, which is the streak requirement of Section III-C doing its work: the per-sample false-positive rate at the operating point is 0.2%, but isolated exceedances never survive three consecutive samples.

The residual false positives are concentrated in two rules and are worth naming rather than rounding away. The impossible-travel rule fires 1.00 times per trial, always on the *legitimate* operator sign-in that follows the attacker’s — correct behaviour that a label-based metric is obliged to count as an error. The command-rate rule fires 1.58 times in the trials where it fires at all, on nominal bursts that cross a per-operator ceiling that is simply set too tightly. The first is a metric artefact; the second is a genuine tuning defect.

B. RQ2: What the baseline adds over spacecraft limits

This is the comparison that matters operationally, because the limit check is what a mission already runs. Figure 3 shows the distribution of warning time across trials: the learned baseline raises a finding 42 ± 2 minutes before the first limit violation (range 37–44 minutes, and in no trial did the limit fire first). Event-level recall for the drift rises from 0.19 to 0.97, and MTTD falls from 47 to 6 minutes.

The interpretation is narrow and we state it narrowly: a limit check tells an operator the spacecraft is *already* outside its

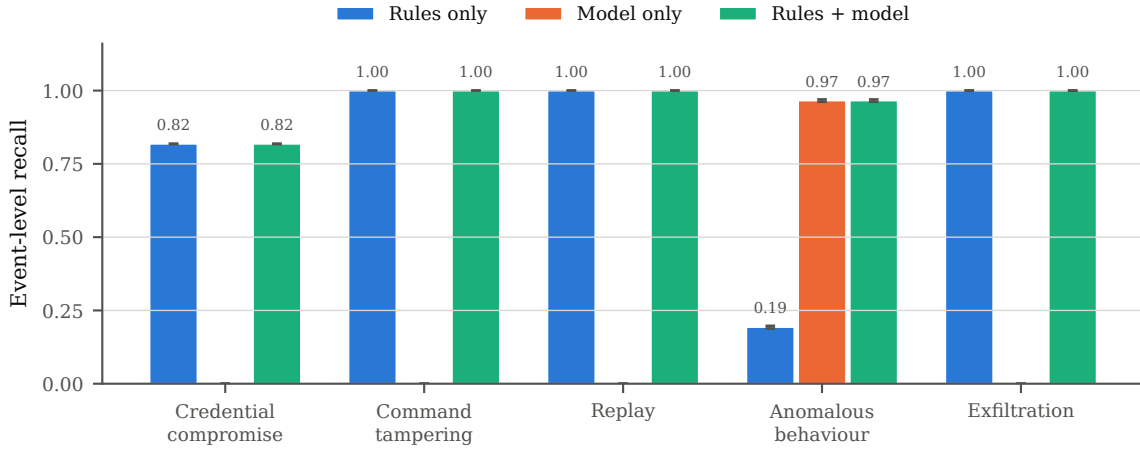


Fig. 2. Event-level recall by scenario and configuration, mean over 20 trials with standard-deviation bars. Absent bars are zero. The rule catalogue and the learned baseline are complementary rather than overlapping: neither configuration alone covers the corpus.

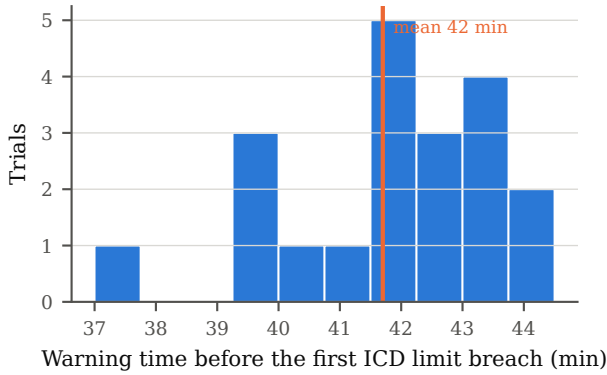


Fig. 3. Warning time the learned baseline provides ahead of the first interface-control limit violation, over 20 trials. The limit check never fired first.

design envelope, whereas the baseline tells them it is heading there. Forty minutes is the difference between diagnosing a degradation and reacting to an alarm. It is not evidence that the model is a good anomaly detector in general — only that it is a better early indicator than a red line.

C. RQ3: Multivariate versus univariate — two negative results

On the injected drift, the covariance buys nothing. Figure 4 shows the receiver operating characteristic of both detectors on the telemetry plane of the attacked runs, zoomed to the only region where they could differ. Over 20 trials the multivariate detector achieves $AUC\ 0.996 \pm 0.001$ and the best single channel 0.998 ± 0.000 . The univariate detector is marginally *better*, and the gap is smaller than the between-trial spread. The rationale we designed the model around — that correlation is what makes a sub-threshold attack visible — is not supported for this attack.

The explanation is in the attack, not the model. The injected drift moves the slew rate, bus current and amplifier temperature several robust standard deviations from their own centres, so each channel is individually anomalous and a per-channel test

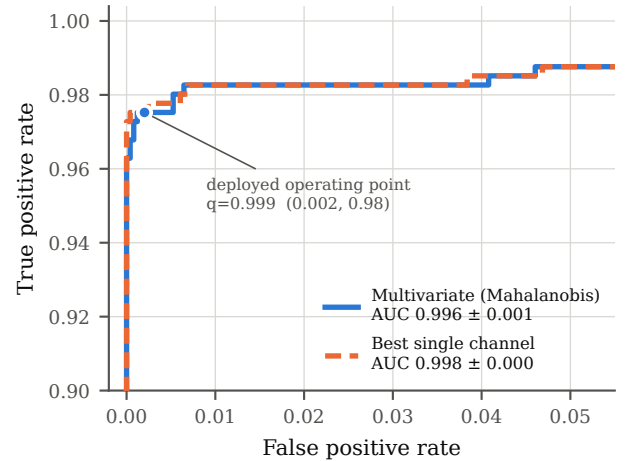


Fig. 4. Telemetry-plane ROC on the attacked runs, zoomed to $FPR \leq 0.055$; over the full range the two curves are indistinguishable. AUC is the mean $\pm$ s.d. over 20 trials. The per-channel detector is marginally better than the multivariate one.

suffices. A fair reading is that our behavioural scenario is easier than intended, and that the experiment does not test the claim.

On an attack that isolates the joint structure, both detectors fail. The phase-swap probe of Section V-E does test it, and the result is worse than inconclusive. Figure 5 shows why: the swapped vectors do not merely fail to exceed the threshold, they score *lower* than nominal telemetry — median squared distance 2.9 against 4.6 for nominal samples. Both detectors are below chance ($AUC\ 0.322 \pm 0.026$ multivariate, 0.385 ± 0.019 univariate) and the attack is flagged in only 1 of 20 trials.

This is a structural limitation, not a tuning problem, and no threshold fixes it. A single Gaussian fitted over an entire orbit learns the *average* relationship between channels, marginalising over the phase; any vector that occurred at any phase is therefore in-distribution. That the swapped samples

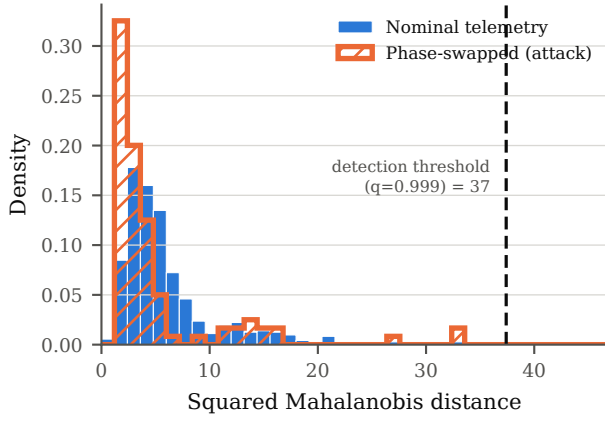


Fig. 5. Score distributions under the phase-swap probe. Replayed housekeeping frames from half an orbit earlier sit *inside* — indeed below — the nominal distribution, because orbit phase is marginalised away by a baseline fitted over a whole orbit. Texture is redundant with colour for grayscale printing.

TABLE V
BASELINE-SIZE SENSITIVITY AT THE DEPLOYED OPERATING POINT ($q = 0.999$). DISCRIMINATION (AUC) BARELY MOVES; THE FALSE-POSITIVE RATE COLLAPSES ONLY ONCE THE TRAINING WINDOW COVERS A FULL DIURNAL CYCLE.

Hours	Samples	τ	AUC	TPR	FPR (%)
1	120	24.6	0.9738	1.000	76.9
2	240	24.5	0.9785	1.000	51.2
4	480	29.8	0.9802	1.000	48.0
8	960	32.9	0.9858	0.993	46.6
12	1440	33.5	0.9887	0.988	42.2
16	1920	34.1	0.9912	0.983	17.6
20	2400	34.8	0.9937	0.975	1.3
24	2880	37.4	0.9944	0.975	0.2

score below the median follows: mid-orbit values are denser than the extremes the true samples occupy. An adversary who replays valid old telemetry hides inside the learned baseline, and makes the spacecraft look healthier than it is.

The fix is to condition on phase rather than to marginalise it: model the residual from a phase-dependent expectation, or fit a mixture indexed by orbit phase, and include phase as an explicit conditioning variable. We report the failure rather than implementing the fix here, because the probe is the contribution — it is a cheap, reusable test that any spacecraft telemetry detector can be subjected to, and we would expect several published detectors to fail it.

D. RQ4: How much clean telemetry the baseline needs

Figure 6 and Table V sweep the size of the training window. The finding is a sharp threshold rather than a gradual improvement: fitting on one hour yields a telemetry false-positive rate of 77%, still 0.2% only once the window reaches a full 24 hours. AUC improves marginally across the sweep (0.974 to 0.994) and true-positive rate declines slightly as the threshold rises, so neither explains the collapse.

The cause is coverage, not sample size. A window shorter than the orbital and diurnal cycle observes only part of the state space — some eclipse fractions, some contact geometries,

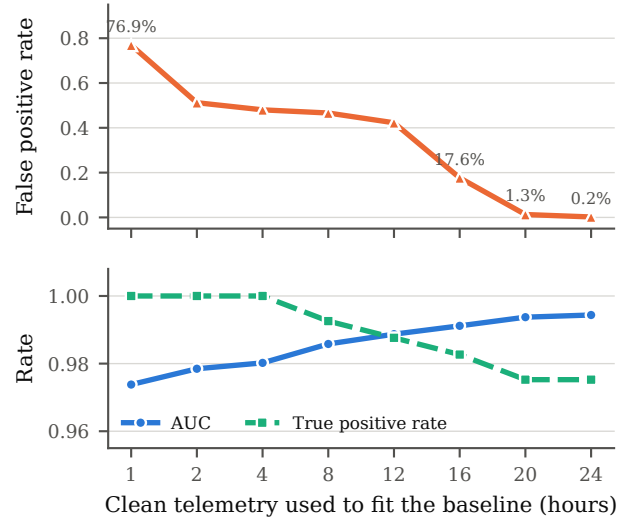


Fig. 6. Baseline-size sensitivity. Upper panel: telemetry false-positive rate at the deployed operating point. Lower panel: AUC and true-positive rate. The false-positive rate collapses only when the training window covers a full diurnal cycle; discrimination barely changes.

TABLE VI
SINGLE-CORE DETECTION COST, BEST OF 5 RUNS OVER A 24-HOUR CORPUS OF 3584 EVENTS.

Configuration	Events/s	μ s/event
Rules only	237 025	4.2
Rules + model	103 205	9.7
Baseline fit: 0.012 s, 2880 samples		
Serialised model: 1.5 KB		

some thermal regimes — so the quantile threshold is calibrated against a distribution the deployed model will not see. The operational rule this implies is concrete: a baseline must span at least one full cycle of every periodicity the mission has, and a detector that reports only AUC would have concluded, wrongly, that one hour of data is nearly as good as a day.

E. RQ5: Cost

Table VI gives the single-core cost. The engine processes 103 205 events per second with the model enabled (9.7 μ s per event) and 237 025 without it; fitting the baseline on a day of telemetry takes 0.01 s and produces a 1.5 KB model. At a 30-second telemetry cadence, one core is approximately five orders of magnitude faster than one station requires, so the detection cost is not the constraint — transport and storage are.

F. Alert volume

Figure 7 plots the analyst-facing load. The mean of 613 findings per simulated day is a misleading summary: the load is extremely bursty, with nothing for twenty hours and a peak of roughly 175 findings in a single hour, of which 26 per day are critical. Aggregate false-positive rates say little about whether a queue is workable, and the base-rate fallacy [17] applies with full force at these volumes. The

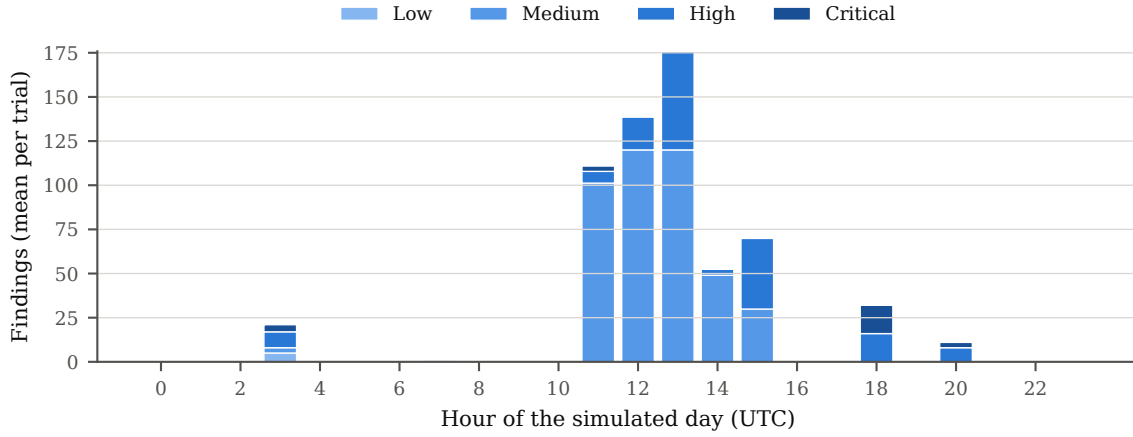


Fig. 7. Findings per hour of the simulated day by severity, mean over 20 trials. The load is bursty rather than steady: the daily mean of 613 findings describes no hour of the day.

practical mitigation in our design is severity discipline plus correlation: critical findings page, and the chains in Table I mean an operator sees one incident rather than four unrelated alerts.

VII. DISCUSSION

A. Rules and models do different jobs

The ablation makes a point that is easy to lose in a field enthusiastic about learned detection: the four scenarios with the highest impact — forged commands, replay, privilege escalation, exfiltration — are caught by deterministic rules, immediately, with no variance and no training data. They are caught because the mission has an explicit policy (an authentication tag must verify; a counter must not repeat; a role may not issue this opcode) and a rule is the natural expression of a policy. Learned detection earns its place only where no policy exists to state: behaviour that is legal at every instant and wrong in aggregate. Treating anomaly detection as a substitute for policy encoding, rather than a complement to it, would have produced a system that misses everything in Figure 2’s first three groups.

B. What the negative results mean for practice

Our two negative results point the same way: the marginal distributions of spacecraft housekeeping channels carry most of the signal that a static baseline can extract, and the joint structure a static baseline learns is too coarse to catch an adversary who respects the marginals. Because the second failure is below chance, it is also *exploitable*: an adversary who knows a Mahalanobis-style baseline is deployed can replay old frames not merely to evade it but to drive the anomaly score down while a fault progresses.

We think the implication generalises beyond our model. Any detector that fits a phase-marginal distribution over a periodic system inherits the same blind spot, including detectors considerably more sophisticated than ours [18], [19] unless they condition on phase explicitly. The phase-swap probe costs a

few lines to implement and we would encourage its use as a standard check.

C. Deployment lessons

The three silent failures in Section IV share a shape: the cloud API accepted the request, returned success, and discarded the data or the configuration intent. A detection pipeline is precisely the kind of system in which such failures are invisible — an empty log group and a quiet queue look identical to a quiet day. We took two lessons. Fail loudly on partial acceptance, even when the API does not consider it an error. And test the cloud path, not only the logic: an in-process emulation of the cloud services caught all three defects in an afternoon, with no credentials and no account.

VIII. LIMITATIONS AND THREATS TO VALIDITY

The corpus is synthetic. This is the dominant threat and we do not minimise it. Our spacecraft model is a sine-wave illumination profile with first-order thermal lags; real house-keeping telemetry has instrument nonlinearity, sensor faults, seasonal beta-angle drift and operator idiosyncrasy that we do not reproduce. Absolute numbers should therefore be read as properties of this corpus, not of a mission. The comparative and structural findings — the rules/model complementarity, the ordering of lead times, the phase-swap blindness, the coverage threshold — depend on the *structure* of the data rather than its fidelity, and we expect them to transfer; the precision figure does not.

Attacks are injected, not executed. No exploit runs against real infrastructure. The adversary is also non-adaptive: it does not observe the detector and adjust, which is the weakest assumption in any static evaluation of this kind [20]. The phase-swap probe is our one concession to an adaptive adversary, and it succeeded.

Label granularity distorts metrics. Recall is per event, so a scenario that labels 404 events and one that labels 7 are not comparable, and the impossible-travel false positive discussed in Section VI is correct detector behaviour scored as an error.

We report per-scenario numbers to keep this visible rather than resolving it.

Scope. One station, one spacecraft, one region. Cross-station correlation — the same source address touching two stations — is a stronger signal than either station alone and we do not evaluate it. Per-container state in the deployed detector (rate windows, anomaly streaks) is shard-sticky rather than guaranteed; only the replay memory is made globally consistent.

The cloud evaluation is emulated. The integration tests run against an in-process emulation of the AWS APIs. It is faithful about the behaviours we depend on, but it is not AWS: it cannot surface authorisation-policy mistakes, service quotas or eventual consistency. The infrastructure code is format-checked and reviewed but has not been applied against a live account, and the cost and latency of the deployed pipeline are therefore unmeasured — only the engine’s compute cost in Table VI is.

Model scope. We evaluate one baseline family. Isolation Forests [21], sequence models [18] and the probabilistic-clustering approach of [19] are the obvious comparisons and are left to future work; the model sits behind a score-and-threshold interface so that they can be substituted and scored on the same labels.

IX. RELATED WORK

Space systems security. Pavur and Martinovic surveyed the field and argued that its central problem is the absence of reproducible artefacts and shared testbeds [3]; this paper is a response to that argument for one part of the system. Pavur et al. demonstrated practical interception of operational VSAT links [2], and Willbold et al. analysed real satellite firmware and found systematic absence of authentication on the commanding path [1]. Falco [12] and Manulis et al. [11] set out security principles for space systems and for the commercial “New Space” segment respectively. SPARTA provides the space-specific adversary taxonomy we map to [5], and ATT&CK the enterprise counterpart [7]. Our focus differs from all of these in being the ground segment’s cloud footprint, and in being evaluated against labels rather than argued.

Spacecraft telemetry anomaly detection. Detecting faults in housekeeping telemetry is a mature field. Hundman et al. applied LSTMs with nonparametric dynamic thresholding to NASA spacecraft channels [18]; Yairi et al. combined probabilistic clustering with dimensionality reduction for satellite housekeeping data [19]; Chandola et al. survey the wider area [22]. This literature targets *faults*, and its evaluations assume anomalies are not adversarially chosen. Our phase-swap result suggests that assumption is load-bearing: a fault will not respect the marginals, and an adversary will.

Detection evaluation methodology. Sommer and Paxson’s critique of machine learning for intrusion detection [20] and Axelsson’s base-rate analysis [17] shape how we report results — per-scenario rather than aggregate, false positives over benign events, and alert volume as a first-class result. Ring et al. survey the network IDS datasets [6] whose absence in this

domain motivated the corpus. Automated adversary emulation in the style of CALDERA [23] informed the composing-injector design.

Standards. The commanding model follows the CCSDS TC space data link protocol [9] and its security layer [10]; our keyed tag and sequence counter stand in for the latter’s authentication and anti-replay services. Findings are emitted in ASFF [8], and the deployment targets the managed ground-station service documented in [24].

X. CONCLUSION

We built an open, labelled testbed for detecting attacks on satellite ground stations, and a cloud-native pipeline that detects five emulated scenarios in every one of 20 trials at 0.997 ± 0.002 precision. The result we consider most useful is not that number. It is that deterministic policy rules, not learned detection, carry the high-impact scenarios; that the learned baseline earns its place on exactly one of the five, where it buys 42 minutes over the spacecraft’s own limits; that its multivariate machinery does not outperform a well-calibrated per-channel detector on that scenario; and that both are blind — below chance — to an adversary who replays valid telemetry from a different orbit phase. The last finding is structural, applies to any phase-marginal baseline, and is testable in a few lines of code.

The artefact is public, deterministic, and runs offline in seconds, so each of these claims can be checked and, we hope, improved on.

AVAILABILITY

The simulator, attack scenarios, detection engine, infrastructure code, experiment harness and this manuscript’s sources are available at <https://github.com/rikosoo/Cloud-Native-Detection-of-Cyberattacks-Against-Satellite-Ground-Station>. Running `python paper/experiments.py` regenerates every number and figure in this paper.

ETHICAL CONSIDERATIONS

No real ground station, spacecraft, or third-party system was touched. All telemetry, credentials and audit records are synthetic; the cryptographic keys in the repository are lab constants, labelled as such, and unsuitable for any operational use. The attack scenarios describe adversary behaviour at the level of detection engineering and contain no exploit code, no vulnerability in any real product, and nothing that shortens the path to attacking an operational mission.

REFERENCES

- [1] J. Willbold, M. Schloegel, M. Vögele, M. Gerhardt, T. Holz, and A. Abbasi, “Space odyssey: An experimental software security analysis of satellites,” in *IEEE Symposium on Security and Privacy (S&P)*, 2023, pp. 1–19.
- [2] J. Pavur, D. Moser, M. Strohmeier, V. Lenders, and I. Martinovic, “A tale of sea and sky: On the security of maritime VSAT communications,” in *IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 1384–1400.
- [3] J. Pavur and I. Martinovic, “SOK: Building a launchpad for impactful satellite cyber-security research,” in *arXiv preprint arXiv:2010.10872*, 2020.

- [4] The White House, “Space policy directive-5: Cybersecurity principles for space systems,” Executive Office of the President of the United States, Tech. Rep., 2020.
- [5] The Aerospace Corporation, “SPARTA: Space attack research and tactic analysis,” <https://sparta.aerospace.org/>, 2024, accessed 2026-09-19.
- [6] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, “A survey of network-based intrusion detection data sets,” *Computers & Security*, vol. 86, pp. 147–167, 2019.
- [7] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, “MITRE ATT&CK: Design and philosophy,” The MITRE Corporation, Tech. Rep. MP180360, 2020.
- [8] Amazon Web Services, “AWS security finding format (ASFF),” <https://docs.aws.amazon.com/securityhub/latest/userguide/securityhub-findings-format.html>, 2026, accessed 2026-09-19.
- [9] Consultative Committee for Space Data Systems, “TC space data link protocol,” CCSDS, Recommended Standard CCSDS 232.0-B-4, 2021.
- [10] —, “Space data link security protocol,” CCSDS, Recommended Standard CCSDS 355.0-B-2, 2022.
- [11] M. Manulis, C. P. Bridges, R. Harrison, V. Sekar, and A. Davis, “Cyber security in new space,” *International Journal of Information Security*, vol. 20, no. 3, pp. 287–311, 2021.
- [12] G. Falco, “Cybersecurity principles for space systems,” *Journal of Aerospace Information Systems*, vol. 16, no. 2, pp. 61–70, 2019.
- [13] P. Syverson, “A taxonomy of replay attacks,” in *IEEE Computer Security Foundations Workshop (CSFW)*, 1994, pp. 187–191.
- [14] P. J. Rousseeuw and C. Croux, “Alternatives to the median absolute deviation,” *Journal of the American Statistical Association*, vol. 88, no. 424, pp. 1273–1283, 1993.
- [15] O. Ledoit and M. Wolf, “A well-conditioned estimator for large-dimensional covariance matrices,” *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004.
- [16] P. C. Mahalanobis, “On the generalised distance in statistics,” *Proceedings of the National Institute of Sciences of India*, vol. 2, no. 1, pp. 49–55, 1936.
- [17] S. Axelsson, “The base-rate fallacy and the difficulty of intrusion detection,” *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205, 2000.
- [18] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, “Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding,” in *ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2018, pp. 387–395.
- [19] T. Yairi, N. Takeishi, T. Oda, Y. Nakajima, N. Nishimura, and N. Takata, “A data-driven health monitoring method for satellite housekeeping data based on probabilistic clustering and dimensionality reduction,” *IEEE Transactions on Aerospace and Electronic Systems*, vol. 53, no. 3, pp. 1384–1401, 2017.
- [20] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *IEEE Symposium on Security and Privacy (S&P)*, 2010, pp. 305–316.
- [21] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [22] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [23] A. Applebaum, D. Miller, B. Strom, C. Korban, and R. Wolf, “Intelligent, automated red team emulation,” in *Annual Conference on Computer Security Applications (ACSAC)*, 2016, pp. 363–373.
- [24] Amazon Web Services, “AWS ground station documentation,” <https://docs.aws.amazon.com/ground-station/>, 2026, accessed 2026-09-19.